



OpenStack Fundamentals: Architecture & Core Concepts

A learning journey for engineers new to OpenStack — from ecosystem basics to production-ready architecture.

What Is OpenStack?

Open-Source Cloud OS

Manages compute, storage, and networking across a data center via a unified API — think AWS, but on your own hardware.

OpenInfra Foundation

100,000+ community members across 180+ countries. Backed by major vendors: Red Hat, Canonical, SUSE, and more.

Release Cadence

Two releases per year (e.g., Zed, Antelope, Bobcat), each supported ~18 months. Every other cycle is a Long-Term Support (SLURP) release.

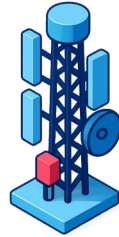


Enterprise Use Cases



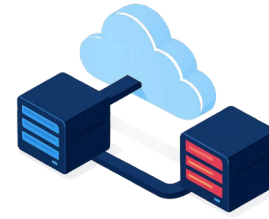
Private Cloud

Telcos, banks, and governments run OpenStack on-premises for full control over their workloads and data sovereignty.



NFV / Telco Cloud

Major carriers like AT&T and Deutsche Telekom virtualize network functions at scale using OpenStack as the foundation.

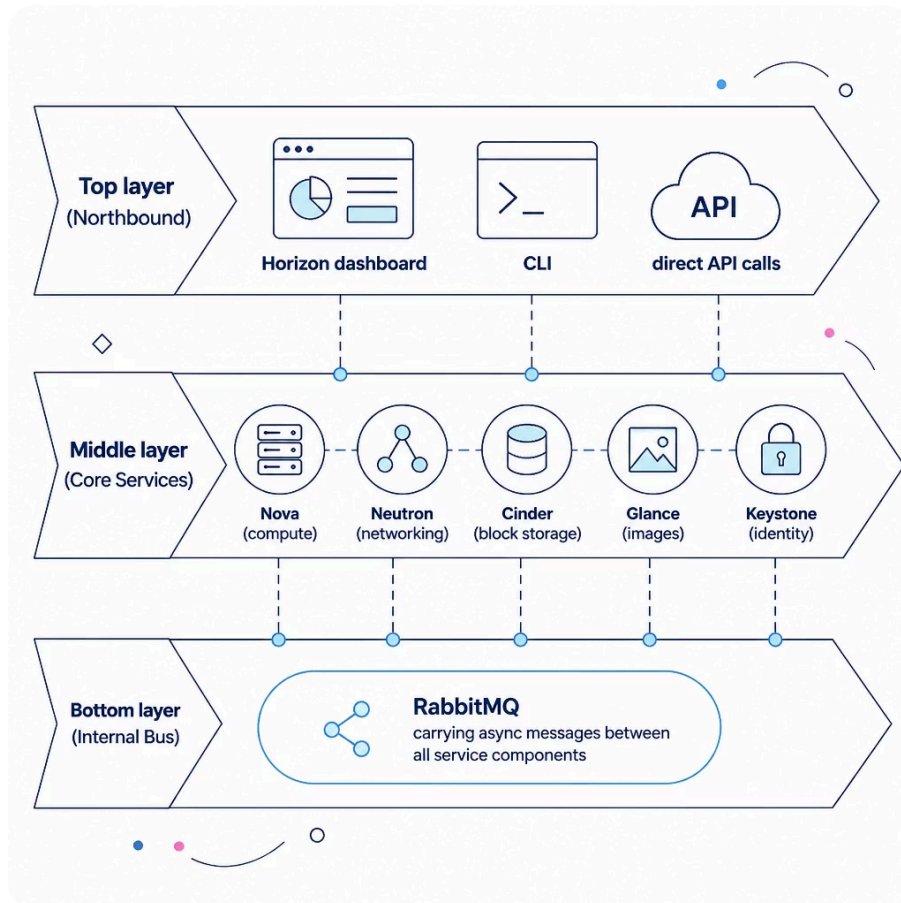


Hybrid Cloud

Bridges on-premises OpenStack deployments with public clouds through standard APIs, enabling consistent workload portability.

Logical Architecture Overview

OpenStack is a collection of loosely coupled services that communicate over REST APIs and a shared message bus.



Northbound Interface

Users and operators interact via **Horizon** (web dashboard), the OpenStack CLI, or direct API calls — all paths lead to the same underlying services.

Core Services

- **Nova** — Compute orchestration
- **Neutron** — Networking
- **Cinder** — Block storage
- **Glance** — Image registry
- **Keystone** — Identity & auth

Internal Bus

RabbitMQ carries async messages between service components, decoupling them from direct dependencies.

Control Plane vs. Data Plane

Control Plane

Handles orchestration, scheduling, and API requests. Runs on **Controller nodes**.

No tenant traffic flows through here — it's purely the brain of the operation.

- ❏ A control plane failure doesn't immediately kill running workloads.

Data Plane

Carries actual **VM traffic**, storage I/O, and tenant network packets. Runs on **Compute**, **Network**, and **Storage** nodes.

Separating the two planes improves resilience — the data plane keeps serving users even when the control plane is degraded.

- VM execution & hypervisor I/O
- Tenant network packet forwarding
- Block and object storage traffic

Controller Node

API & Scheduler Hub

Hosts all API endpoints, schedulers, and the Horizon dashboard for every OpenStack service.

Critical Supporting Services

MariaDB — state & config DB

RabbitMQ — message queue

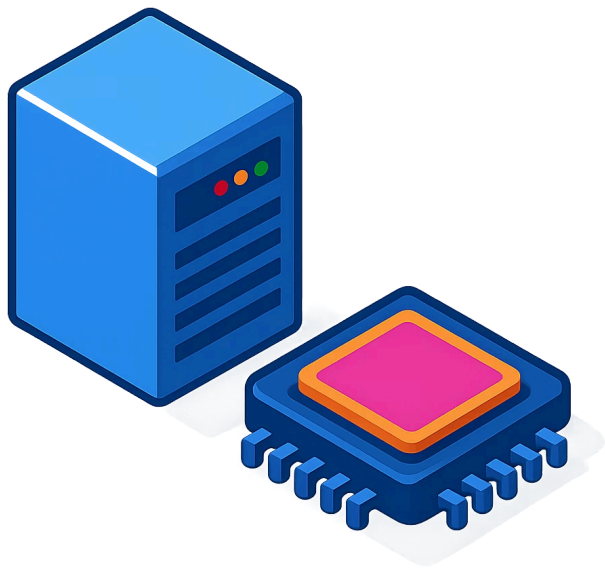
Memcached — token & session cache

HA Deployment

Typically a **3-node cluster** using Pacemaker and Galera replication to eliminate single points of failure.



Compute & Network Nodes



Compute Node

Runs the **Nova-compute** agent alongside a hypervisor — **KVM/QEMU** by default. This is where virtual machines actually execute and consume CPU, RAM, and local disk.

Network Node

Runs **Neutron agents** handling routing, floating IPs, security groups, and DHCP for tenant networks.

Modern Deployments

Co-locating network functions on compute nodes using **OVN** (Open Virtual Network) is now common practice — eliminating the need for a dedicated network node and reducing infrastructure overhead.

Storage Node & Placement Service



Block Storage (Cinder)

Provides persistent volumes backed by **LVM** or **Ceph**. Decoupled from compute — volumes survive VM deletion and can be reattached to new instances.



Object Storage (Swift)

Horizontally scalable, eventually consistent object store. Ideal for unstructured data like backups, images, and media files at petabyte scale.



Placement API

Tracks resource inventories — **vCPU, RAM, disk, and custom traits** — across all nodes. Nova's scheduler queries Placement to find the optimal host for every VM launch.

Supporting Services at a Glance



RabbitMQ

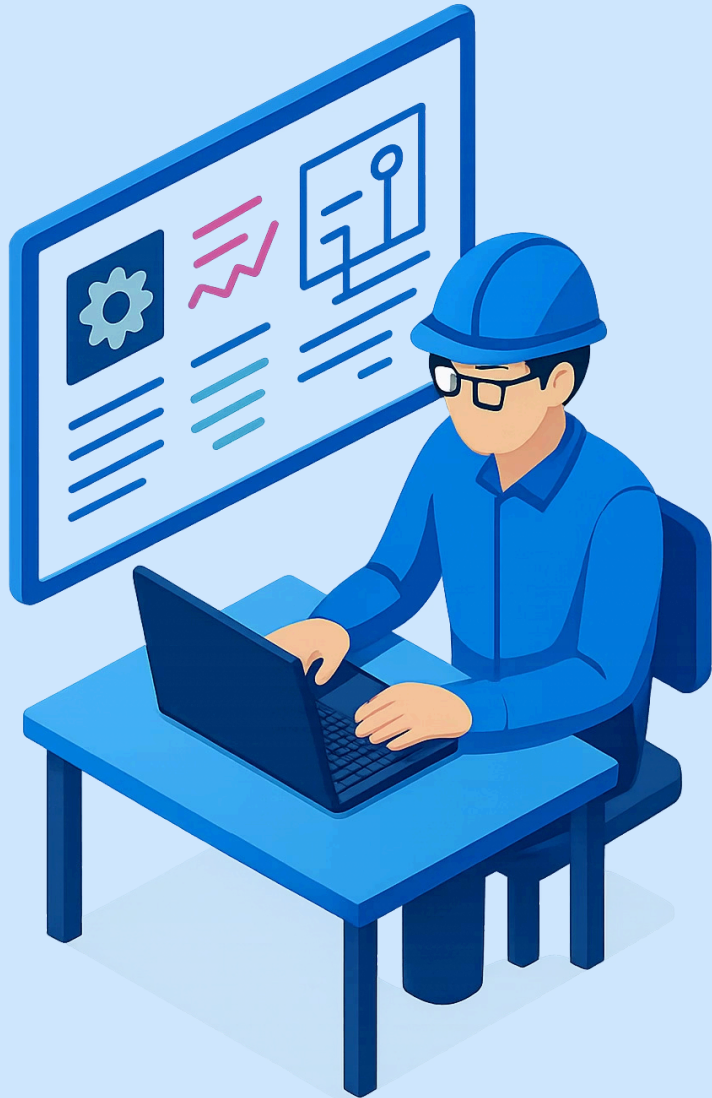
AMQP message broker that decouples service components — they communicate asynchronously without direct dependencies on each other.

MariaDB (Galera)

Stores all service state: instances, networks, volumes, users. Multi-master Galera replication ensures high availability with no single point of failure.

Memcached

Caches Keystone tokens in memory, dramatically reducing database load on every API call across the entire cluster.



Your OpenStack Learning Path



Step 1 — Lab It Up

Deploy a single-node all-in-one with **DevStack** or **Microstack** to explore the APIs hands-on in a safe environment.



Step 2 — Study the Services

Follow the OpenInfra docs in order: **Keystone** → **Nova** → **Neutron** → **Cinder**. Each service builds on the last.



Step 3 — Go Production

Use **TripleO**, **Kolla-Ansible**, or **OpenStack-Ansible** for multi-node deployments with HA and proper network segmentation.